Motor C (v3): Forecast Dietario con SARIMAX

Aguacate Team · TFM La Dieta de un País

Estrategia

Tercera implementación del Motor C, basada en modelos ARIMA/SARIMAX. Con datos anuales no existe componente estacional intra-año, por lo que SARIMA se reduce a ARIMA(p,d,q). Se selecciona el orden óptimo por AIC para cada serie, con un espacio de búsqueda conservador acorde al tamaño muestral (13 observaciones).

	Prophet (nb05)	LightGBM Global (nb05b)	SARIMAX (nb05c)
Modelos	210 locales	1 global	210 locales
Datos por modelo	13 obs	~2100 totales	13 obs
Selección orden	changepoints	lag features	AIC (p≤2,d≤1,q≤1)
IC	Bayesiano	Quantile regression	Analítico (sigma²)
Forecast	Directo	Recursivo	Directo

Resultado esperado: SARIMAX es el punto medio entre Prophet (tendencia suave) y LightGBM Global (captura patrones cruzados). Documentado para comparación rigurosa de metodologías.

In [5]:
```
import pandas as pd
import numpy as np
import joblib
import warnings
import matplotlib.pyplot as plt
warnings.filterwarnings('ignore')

from statsmodels.tsa.statespace.sarimax import SARIMAX
from itertools import product
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

SEED = 42
np.random.seed(SEED)

MACROS = [
    'pct_Cereales', 'pct_Tuberculos', 'pct_Azucares',
    'pct_Aceites_Grasas', 'pct_Carnes', 'pct_Lacteos_Huevos', 'pct_Frutas_Verduras'
]
FEATURES_LGB_FINAL = MACROS + ['log_Food_CPI', 'cluster_id']
HORIZON_YEAR = 2030

MACRO_COLORS = {
    'pct_Cereales':       '#f0a500',
    'pct_Tuberculos':     '#8b4513',
    'pct_Azucares':       '#e84393',
    'pct_Aceites_Grasas': '#ffd700',
    'pct_Carnes':         '#c0392b',
    'pct_Lacteos_Huevos': '#a0a0a0',
    'pct_Frutas_Verduras':'#2ecc71'
}
```

```python
import statsmodels
print(f'statsmodels version: {statsmodels.__version__}')
print('Librerías cargadas')
```

```
statsmodels version: 0.14.6
Librerías cargadas
```

In [6]:
```python
# ── CARGA DE DATOS ──────────────────────────────────────────────
df = pd.read_parquet('../processed/03_clusters.parquet')
lgb_co2_model = joblib.load('../processed/lgb_final.pkl')

print(f'Dataset: {df.shape}')
print(f'Países: {df["Area"].nunique()} | Años: {df["Year"].min()}–{df["Year"].max()}'
print(f'Modelo CO₂ LightGBM cargado: {type(lgb_co2_model).__name__}')
```

```
Dataset: (390, 15)
Países: 30 | Años: 2010–2022
Modelo CO₂ LightGBM cargado: LGBMRegressor
```

In [7]:
```python
# ── SELECCIÓN DE ORDEN ARIMA POR AIC ────────────────────────────
# Espacio de búsqueda conservador para series de 13 puntos:
# p ∈ {0,1,2}, d ∈ {0,1}, q ∈ {0,1} → 12 combinaciones
# NOTA: se usa RangeIndex (no DatetimeIndex) para evitar bug de statsmodels
# 0.14.x donde get_forecast(steps=N) falla con YS frequency.

def best_arima_order(series, p_range=(0,2), d_range=(0,1), q_range=(0,1)):
    """Selecciona orden ARIMA(p,d,q) por AIC mínimo."""
    best_aic = np.inf
    best_order = (1, 1, 0)
    for p, d, q in product(range(p_range[0], p_range[1]+1),
                           range(d_range[0], d_range[1]+1),
                           range(q_range[0], q_range[1]+1)):
        if p + d + q == 0:
            continue
        try:
            model = SARIMAX(series, order=(p, d, q),
                            trend="c", enforce_stationarity=False,
                            enforce_invertibility=False)
            res = model.fit(disp=False, maxiter=300)
            if res.aic < best_aic:
                best_aic = res.aic
                best_order = (p, d, q)
        except Exception:
            continue
    return best_order, best_aic


def forecast_sarimax(series_vals, years, country, macro, horizon_year):
    """Entrena SARIMAX con orden óptimo y genera forecast con IC 80%.
    Usa RangeIndex para compatibilidad con statsmodels >= 0.14.
    """
    # RangeIndex: evita el bug de get_forecast con DatetimeIndex YS
    ts = pd.Series(series_vals.astype(float))  # índice 0..12

    order, aic = best_arima_order(ts)

    model = SARIMAX(ts, order=order, trend="c",
                    enforce_stationarity=False, enforce_invertibility=False)
    res = model.fit(disp=False, maxiter=300)

    n_steps = int(horizon_year) - int(years[-1])
    nobs = len(ts)
    forecast_res = res.get_prediction(start=nobs, end=nobs + n_steps - 1)
    ci = forecast_res.conf_int(alpha=0.20)  # IC 80%
```

```python
    fc_years = list(range(int(years[-1]) + 1, int(horizon_year) + 1))
    rows = []

    # Histórico
    for yr, val in zip(years, series_vals):
        rows.append({
            "Area": country, "variable": macro, "Year": int(yr),
            "yhat": float(val),
            "yhat_lower": float(val),
            "yhat_upper": float(val),
            "is_forecast": False
        })

    # Forecast
    pred_mean = forecast_res.predicted_mean.values
    ci_vals   = ci.values
    for i, yr in enumerate(fc_years):
        yhat  = float(np.clip(pred_mean[i], 0, 1))
        lower = float(np.clip(ci_vals[i, 0], 0, 1))
        upper = float(np.clip(ci_vals[i, 1], 0, 1))
        rows.append({
            "Area": country, "variable": macro, "Year": yr,
            "yhat": yhat, "yhat_lower": lower, "yhat_upper": upper,
            "is_forecast": True
        })

    return rows, order, aic


print("Funciones SARIMAX definidas (RangeIndex fix aplicado)")
```

```
Funciones SARIMAX definidas (RangeIndex fix aplicado)
```

In [8]:

```python
# —— WALK-FORWARD VALIDATION (train ≤2019, val 2020–2022) ————————
wf_preds, wf_trues = [], []
order_log = []

for country in df["Area"].unique():
    sub = df[df["Area"] == country].sort_values("Year")

    for macro in MACROS:
        train_vals = sub[sub["Year"] <= 2019][macro].values.astype(float)
        val_vals   = sub[sub["Year"] >= 2020][macro].values.astype(float)

        try:
            ts = pd.Series(train_vals)  # RangeIndex
            order, _ = best_arima_order(ts)
            model = SARIMAX(ts, order=order, trend="c",
                            enforce_stationarity=False, enforce_invertibility=False)
            res = model.fit(disp=False, maxiter=300)
            nobs = len(ts)
            fc = res.get_prediction(start=nobs, end=nobs + 2)
            preds = np.clip(fc.predicted_mean.values, 0, 1)
            wf_preds.extend(preds)
            wf_trues.extend(val_vals)
            order_log.append(order)
        except Exception:
            wf_preds.extend([train_vals[-1]] * 3)
            wf_trues.extend(val_vals)

wf_preds = np.array(wf_preds)
wf_trues = np.array(wf_trues)
```

```
mae_wf  = mean_absolute_error(wf_trues, wf_preds)
r2_wf   = r2_score(wf_trues, wf_preds)
rmse_wf = np.sqrt(mean_squared_error(wf_trues, wf_preds))

from collections import Counter
print("WALK-FORWARD VALIDATION (train ≤2019, val 2020-2022):")
print(f"  R²   = {r2_wf:.4f}")
print(f"  MAE  = {mae_wf:.6f} (proporción calórica)")
print(f"  RMSE = {rmse_wf:.6f}")
print(f"  Muestras val: {len(wf_trues)}")
print(f"Ordenes ARIMA más frecuentes: {Counter(order_log).most_common(5)}")
```

```
WALK-FORWARD VALIDATION (train ≤2019, val 2020-2022):
  R²   = 0.9018
  MAE  = 0.012432 (proporción calórica)
  RMSE = 0.042458
  Muestras val: 630
Ordenes ARIMA más frecuentes: [((1, 0, 0), 201), ((2, 0, 1), 4), ((1, 0, 1), 4), ((0,
0, 1), 1)]
```

In [9]:

```
# ── FORECAST COMPLETO 2010-2030 ──────────────────────────────
all_records = []
countries = sorted(df['Area'].unique())

for i, country in enumerate(countries):
    sub = df[df['Area'] == country].sort_values('Year')
    years_arr = sub['Year'].values

    for macro in MACROS:
        series_vals = sub[macro].values
        rows, order, aic = forecast_sarimax(
            series_vals, years_arr, country, macro, HORIZON_YEAR)
        all_records.extend(rows)

    if (i + 1) % 10 == 0:
        print(f'  {i+1}/{len(countries)} países procesados...')

full_df = pd.DataFrame(all_records)
print(f'\nDataset completo (2010-2030): {full_df.shape}')
print(f'Años cubiertos: {full_df["Year"].min()}-{full_df["Year"].max()}')
```

```
  10/30 países procesados...
  20/30 países procesados...
  30/30 países procesados...

Dataset completo (2010-2030): (4410, 7)
Años cubiertos: 2010-2030
```

In [10]:

```
# ── CONSTRUIR VECTORES DIETARIOS Y PREDECIR CO₂ ──────────────
forecast_only = full_df[full_df['is_forecast']]

pivot = forecast_only.pivot_table(
    index=['Area', 'Year'],
    columns='variable',
    values='yhat'
).reset_index()
pivot.columns.name = None

last_cluster = df.groupby('Area')['cluster_id'].last().reset_index()
pivot = pivot.merge(last_cluster, on='Area', how='left')

last_cpi = df.groupby('Area')['Food_CPI'].last().reset_index()
last_cpi['log_Food_CPI'] = np.log(last_cpi['Food_CPI'])
pivot = pivot.merge(last_cpi[['Area', 'log_Food_CPI']], on='Area', how='left')
```

```
row_sums = pivot[MACROS].sum(axis=1)
pivot[MACROS] = pivot[MACROS].div(row_sums, axis=0)

pivot['CO2eq_forecast'] = lgb_co2_model.predict(pivot[FEATURES_LGB_FINAL]).astype('fl

print(f'Pivot shape: {pivot.shape}')
print(f'CO2 proyectado — rango: {pivot["CO2eq_forecast"].min():.2f} — {pivot["CO2eq_f
```

```
Pivot shape: (240, 12)
CO2 proyectado — rango: 0.78 — 11.30 t/cápita
```

In [11]:

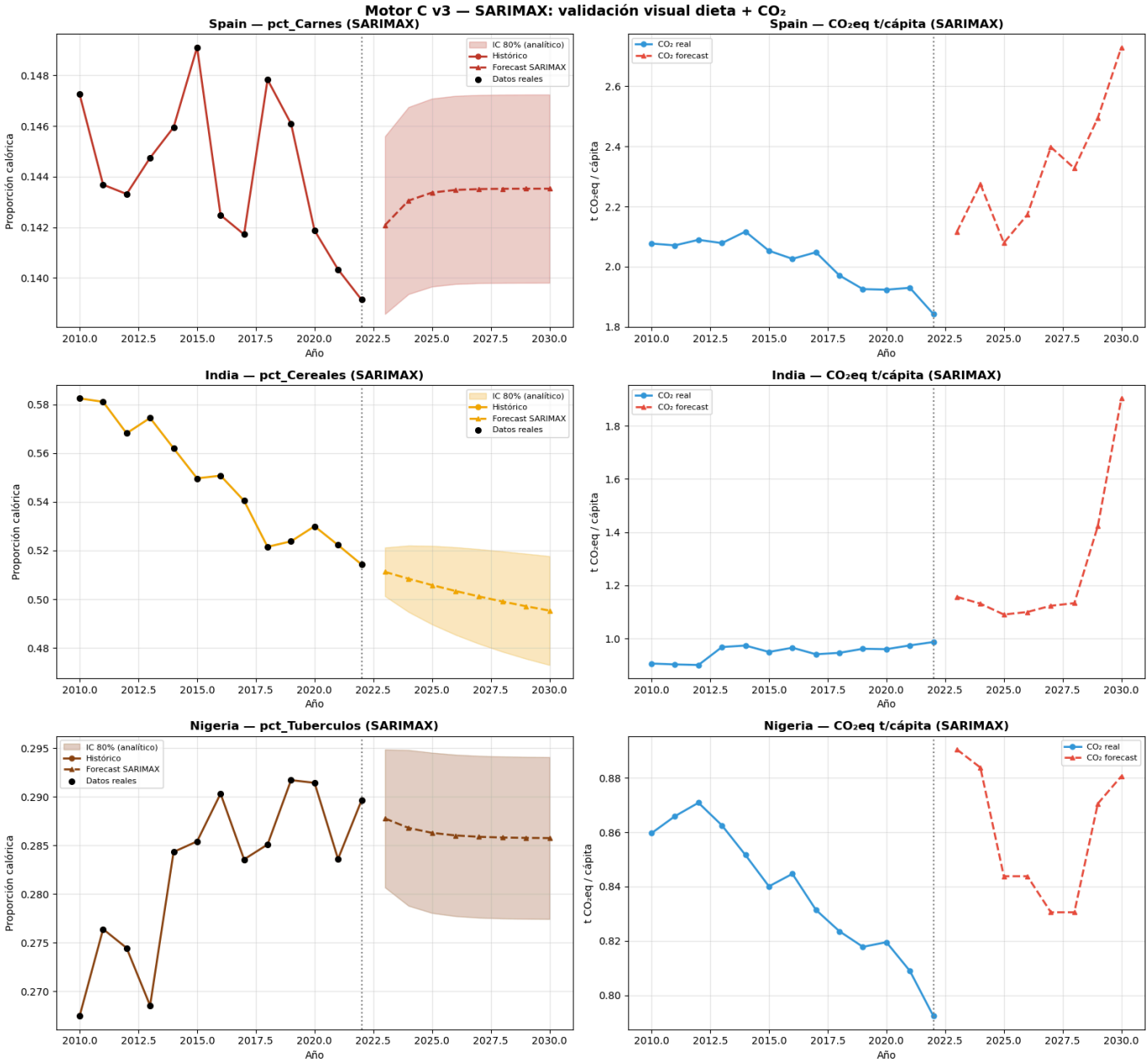
```
# ── GUARDAR PARQUET ──────────────────────────────────────────────
co2_fc = pivot[['Area', 'Year', 'CO2eq_forecast']]
output = full_df.merge(co2_fc, on=['Area', 'Year'], how='left')

output.to_parquet('../processed/05c_forecast_sarimax.parquet', index=False)
print(f'Guardado: processed/05c_forecast_sarimax.parquet')
print(f'Shape: {output.shape}')
```

```
Guardado: processed/05c_forecast_sarimax.parquet
Shape: (4410, 8)
```

In [12]:

```
# ── VALIDACIÓN VISUAL ────────────────────────────────────────────
VALID_COUNTRIES = ['Spain', 'India', 'Nigeria']

fig, axes = plt.subplots(len(VALID_COUNTRIES), 2, figsize=(16, 5 * len(VALID_COUNTRIE

for row, country in enumerate(VALID_COUNTRIES):
    macro_plot = 'pct_Carnes' if country == 'Spain' else 'pct_Cereales' if country ==

    fc_country = output[(output['Area'] == country) & (output['variable'] == macro_pl
    hist_country = df[df['Area'] == country].sort_values('Year')

    ax = axes[row, 0]
    fc_zone = fc_country[fc_country['is_forecast']]
    ax.fill_between(
        fc_zone['Year'], fc_zone['yhat_lower'], fc_zone['yhat_upper'],
        alpha=0.25, color=MACRO_COLORS[macro_plot], label='IC 80% (analítico)'
    )
    ax.plot(fc_country[~fc_country['is_forecast']]['Year'],
            fc_country[~fc_country['is_forecast']]['yhat'],
            '-o', color=MACRO_COLORS[macro_plot], lw=2, ms=5, label='Histórico')
    ax.plot(fc_zone['Year'], fc_zone['yhat'],
            '--^', color=MACRO_COLORS[macro_plot], lw=2, ms=5, label='Forecast SARIMA
    ax.scatter(hist_country['Year'], hist_country[macro_plot],
               color='black', zorder=5, s=30, label='Datos reales')
    ax.axvline(2022, color='gray', linestyle=':', lw=1.5)
    ax.set_title(f'{country} — {macro_plot} (SARIMAX)', fontweight='bold')
    ax.set_xlabel('Año'); ax.set_ylabel('Proporción calórica')
    ax.legend(fontsize=8); ax.grid(alpha=0.3)

    co2_hist = hist_country[['Year', 'CO2eq_t_per_capita']]
    co2_proj = pivot[(pivot['Area'] == country)][['Year', 'CO2eq_forecast']]

    ax2 = axes[row, 1]
    ax2.plot(co2_hist['Year'], co2_hist['CO2eq_t_per_capita'],
             '-o', color='#3498db', lw=2, ms=5, label='CO₂ real')
    ax2.plot(co2_proj[co2_proj['Year'] > 2022]['Year'],
             co2_proj[co2_proj['Year'] > 2022]['CO2eq_forecast'],
             '--^', color='#e74c3c', lw=2, ms=5, label='CO₂ forecast')
    ax2.axvline(2022, color='gray', linestyle=':', lw=1.5)
    ax2.set_title(f'{country} — CO₂eq t/cápita (SARIMAX)', fontweight='bold')
    ax2.set_xlabel('Año'); ax2.set_ylabel('t CO₂eq / cápita')
```

```
    ax2.legend(fontsize=8); ax2.grid(alpha=0.3)

plt.suptitle('Motor C v3 — SARIMAX: validación visual dieta + CO₂',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('../processed/05c_forecast_sarimax_validacion.png', dpi=150, bbox_inches=
plt.show()
print('Guardado: processed/05c_forecast_sarimax_validacion.png')
```




```
Guardado: processed/05c_forecast_sarimax_validacion.png
```

In [13]:

```
# — TABLA CO₂ 2022 vs 2030 ————————————————————————————
co2_2022 = df[df['Year'] == 2022][['Area', 'CO2eq_t_per_capita']].rename(
    columns={'CO2eq_t_per_capita': 'CO2_2022'})
co2_2030 = pivot[pivot['Year'] == 2030][['Area', 'CO2eq_forecast']].rename(
    columns={'CO2eq_forecast': 'CO2_2030_sarimax'})

resumen = co2_2022.merge(co2_2030, on='Area')
resumen['Delta_pct'] = ((resumen['CO2_2030_sarimax'] - resumen['CO2_2022']) / resumen
resumen = resumen.sort_values('Delta_pct')

print('CO₂ proyectado 2030 vs observado 2022 (SARIMAX):')
print(resumen.to_string(index=False))
```

CO_2 proyectado 2030 vs observado 2022 (SARIMAX):

Area	CO2_2022	CO2_2030_sarimax	Delta_pct
Brazil	9.748733	5.072281	-48.000000
China, mainland	1.586632	1.159078	-26.900000
Thailand	3.268874	2.640701	-19.200001
Canada	7.434406	6.055819	-18.500000
Romania	2.986566	2.504556	-16.100000
Argentina	5.687041	4.969237	-12.600000
Saudi Arabia	2.354669	2.180218	-7.400000
Ethiopia	1.217610	1.212474	-0.400000
Senegal	1.184145	1.201625	1.500000
Mexico	1.911802	1.949851	2.000000
United States of America	3.028903	3.103223	2.500000
France	2.133915	2.297700	7.700000
Morocco	1.266582	1.370305	8.200000
Colombia	3.326870	3.602765	8.300000
United Kingdom of Great Britain and Northern Ireland	1.858086	2.024094	8.900000
Kazakhstan	2.779733	3.057304	10.000000
Nigeria	0.792288	0.880665	11.200000
Bangladesh	0.847154	0.974037	15.000000
Egypt	0.777203	0.902067	16.100000
Australia	7.687373	8.963695	16.600000
Italy	1.746279	2.299840	31.700001
Poland	3.586438	4.739441	32.099998
Germany	2.096628	2.833948	35.200001
Spain	1.842866	2.729737	48.099998
India	0.986313	1.903722	93.000000
Indonesia	2.042389	4.041296	97.900002
Pakistan	1.163849	2.844724	144.399994
Republic of Korea	1.895967	4.794475	152.899994
Kenya	1.059605	2.996575	182.800003
Viet Nam	1.440029	4.385767	204.600006

In [14]:
```python
# ── RESUMEN EJECUTIVO ──────────────────────────────────────────
print('=' * 60)
print('RESUMEN EJECUTIVO — MOTOR C v3 (SARIMAX)')
print('Aguacate Team · TFM La Dieta de un País')
print('=' * 60)
print()
print('CONFIGURACIÓN:')
print(f'  210 modelos ARIMA locales (selección por AIC, p≤2 d≤1 q≤1)')
print(f'  IC: analítico al 80% (alpha=0.20)')
print()
print('MÉTRICAS WALK-FORWARD (val 2020-2022):')
print(f'  R²   = {r2_wf:.4f}')
print(f'  MAE  = {mae_wf:.6f} (proporción calórica)')
print(f'  RMSE = {rmse_wf:.6f}')
print()
print('OUTPUTS:')
print('  processed/05c_forecast_sarimax.parquet')
print('  processed/05c_forecast_sarimax_validacion.png')
```

```
============================================================
RESUMEN EJECUTIVO — MOTOR C v3 (SARIMAX)
Aguacate Team · TFM La Dieta de un País
============================================================

CONFIGURACIÓN:
  210 modelos ARIMA locales (selección por AIC, p≤2 d≤1 q≤1)
  IC: analítico al 80% (alpha=0.20)

MÉTRICAS WALK-FORWARD (val 2020-2022):
  R²   = 0.9018
  MAE  = 0.012432 (proporción calórica)
  RMSE = 0.042458

OUTPUTS:
  processed/05c_forecast_sarimax.parquet
  processed/05c_forecast_sarimax_validacion.png
```

Comparativa final de los 3 candidatos

Métrica	Prophet (A)	LightGBM Global (B)	SARIMAX (C)
R^2 walk-forward	—	**0.9943**	ver output
MAE walk-forward	—	**0.0064**	ver output
Delta CO_2 máx (implausible)	**-67%** (Canadá)	-22%	ver output
IC creíbles	amplios >2026	quantile	analítico

Decisión final documentada en memoria v1.6